

Unit 1

Introduction to Linux

1. Linux is a family of open-source Unix-like operating systems based on the Linux kernel.
 2. Created by Linus Torvalds in 1991, it has since grown into a large community-driven project.
 3. Linux is known for its robustness, security, and versatility, making it a popular choice for a wide range of applications, from servers and desktops to embedded systems.
 4. The source code is freely available to anyone, allowing users to study, modify, and distribute it.
 5. Multiple users can access the system simultaneously without interfering with each other.
 6. Linux can execute multiple processes at the same time.
 7. Linux can run on a wide variety of hardware platforms.
 8. Linux systems are known for their long uptime and minimal need for reboots.
-

Linux Distributions

Ubuntu

- **Target Audience:** Beginners, desktop users, and developers.
- **Features:**
 - User-friendly interface with a focus on ease of use.
 - Regular releases every six months, with Long Term Support (LTS) versions available every two years, offering five years of support.
 - Large community and extensive documentation.
 - Software Center for easy application installation.
 - Derivatives like Kubuntu (KDE), Xubuntu (Xfce), and Lubuntu (LXDE) for different desktop environments.

2. Fedora

- **Target Audience:** Developers and users wanting the latest features.
- **Features:**
 - Sponsored by Red Hat, often serving as a testing ground for features that later appear in Red Hat Enterprise Linux (RHEL).
 - Known for integrating the latest software and technologies.
 - Uses the GNOME desktop environment by default.
 - Short release cycle with support for each version lasting around 13 months.

3. Debian

- **Target Audience:** Users seeking stability and a robust server or desktop environment.
- **Features:**
 - Known for its stability and reliability.
 - Large software repository with over 50,000 packages.
 - Three main branches: Stable, Testing, and Unstable, catering to different needs for stability and new features.

- Basis for many other distributions, including Ubuntu.

4. Arch Linux

- **Target Audience:** Advanced users who prefer a do-it-yourself approach.
- **Features:**
 - Rolling release model, meaning continuous updates without the need for major upgrades.
 - Minimalistic and highly customizable, starting with a bare-bones installation.
 - Strong emphasis on simplicity and control.
 - Excellent documentation through the Arch Wiki.

5. CentOS

- **Target Audience:** Enterprise environments and servers.
- **Features:**
 - Community-supported and functionally compatible with Red Hat Enterprise Linux (RHEL).
 - Known for its stability and long-term support, making it ideal for servers and critical applications.
 - CentOS Stream offers a rolling-preview of the next RHEL minor release, providing a midway point between Fedora and RHEL.

6. openSUSE

- **Target Audience:** Developers, system administrators, and desktop users.
- **Features:**
 - Two main branches: openSUSE Leap (regular release, stable) and openSUSE Tumbleweed (rolling release, cutting-edge).
 - YaST (Yet another Setup Tool) for easy system configuration and management.
 - Strong focus on stability and professional use.

7. Linux Mint

- **Target Audience:** Users migrating from Windows, and those seeking a polished desktop experience.
- **Features:**
 - Based on Ubuntu, offering the same stability with additional tweaks and improvements.
 - Cinnamon, MATE, and Xfce desktop environments, with Cinnamon being the most popular.
 - Focuses on providing a familiar and user-friendly interface.
 - Out-of-the-box multimedia support.

8. Manjaro

- **Target Audience:** Users who want the benefits of Arch Linux with a more user-friendly setup.
- **Features:**
 - Based on Arch Linux but with a simplified installation process.
 - Offers different desktop environments such as XFCE, GNOME, and KDE.
 - Rolling release model with a focus on stability and user-friendliness.
 - Pre-installed codecs and drivers for a better out-of-the-box experience.

9. Elementary OS

- **Target Audience:** Users seeking a beautiful and minimalist desktop environment.
- **Features:**
 - Focus on design and user experience, resembling macOS.
 - Pantheon desktop environment, developed in-house.
 - Minimalistic approach with a curated set of applications.
 - Strong emphasis on privacy and simplicity.

10. Kali Linux

- **Target Audience:** Security professionals and ethical hackers.
- **Features:**

- Based on Debian, specifically tailored for penetration testing and security auditing.
- Pre-installed with numerous security tools and utilities.
- Frequent updates to include the latest security tools.
- Not recommended for general desktop use due to its specialized nature.

Note: Watch [this](#) short video to understand a bit deeper in to different types of Linux distros.

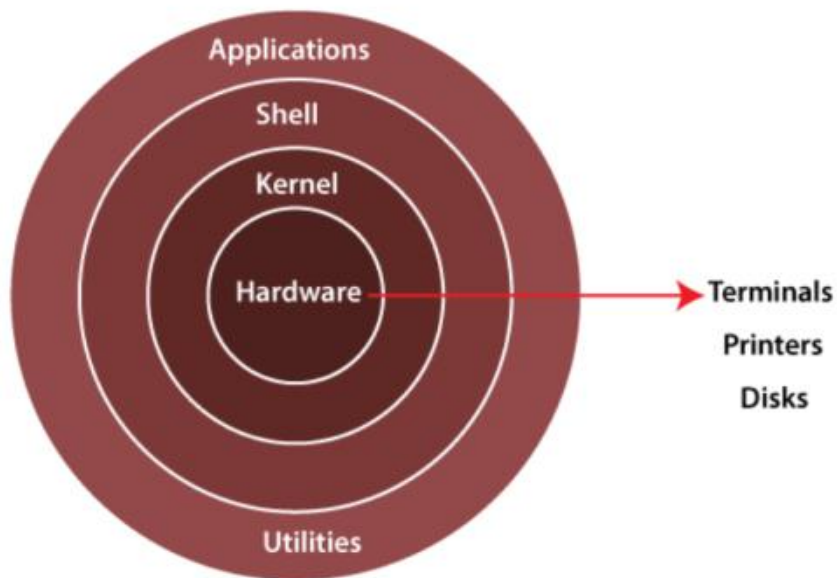
GNOME

1. GNOME, or GNU Network Object Model Environment, is a free and open-source desktop environment for Linux and other Unix-like operating systems.
 2. It's designed to be easy to use, even for non-programmers, and to provide a focused working environment.
 3. GNOME is similar to the Windows desktop interface and includes applications like a word processor, spreadsheet program, database manager, presentation developer, web browser, email program, and more.
 4. GNOME is the default desktop environment in some popular free and open source Linux-based operating systems, such as Fedora, Debian, and Ubuntu.
 5. Ubuntu Desktop comes with a slightly modified version of GNOME.
 6. Some features of GNOME include:
 - Side-by-side windows: Allows users to view multiple documents at once.
 - Online account integration: Allows users to access all their data in one place.
 - Messaging system: Deals with notifications so users can respond quickly or return to them later.
 - Visual effects: Makes use of hardware acceleration support provided by Clutter, an OpenGL-based graphics library.
-

KDE

1. KDE (K Desktop Environment) is a highly customizable and visually appealing desktop environment for Linux and other Unix-like systems.
 2. Known for its flexibility, KDE offers a wide range of configuration options and a rich set of features.
 3. The flagship application of KDE is the Plasma desktop, which provides a sleek and modern interface.
 4. KDE includes a suite of integrated applications like Dolphin (file manager), Konsole (terminal emulator), and Kontact (personal information manager).
 5. It emphasizes productivity and efficiency, offering features like virtual desktops, widgets, and a powerful search tool.
 6. KDE is suitable for both new and experienced users, supporting extensive customization while maintaining user-friendliness.
 7. Notable distributions featuring KDE as the default environment include Kubuntu and openSUSE.
-

Architecture of linux OS



1. Kernel

The kernel is the core part of the Linux operating system. It manages hardware resources and provides essential services to the rest of the system.

2. System Libraries

System libraries provide a set of standard functions that programs can use to interact with the kernel and perform various operations.

3. System Utilities

System utilities are essential programs that perform system-related tasks. These can be divided into:

- Basic Utilities: Essential tools for file manipulation, text processing, and system monitoring.
- Administrative Utilities: Tools for system administration tasks such as managing users, configuring hardware, and maintaining the system.

4. Shell

The shell is a command-line interpreter that allows users to interact with the system by typing commands.

5. User Applications

User applications are the software programs that users run to perform various tasks. These can be anything from text editors to web browsers.

Features of Linux

1. Open source: The source code is available for anyone to explore and modify.
 2. Security: Linux offers robust security features like encryption, security boot support, and access controls.
 3. Stability: Linux systems can run for years without issues or downtime.
 4. Scalability: Linux can run on a number of hardware platforms, from small embedded devices to large-scale server clusters.
 5. Flexibility: Linux is highly customizable, allowing users to tailor the operating system to meet their needs.
 6. Package management: Linux has a powerful package management system that makes it easy to install, update, and remove software packages.
 7. File system: Linux supports different types of file systems, like xfs and ext4.
 8. Graphical user interface (GUI): Linux has a GUI.
 9. Shell/command-line interface: Linux has a shell/command-line interface.
 10. Customized keyboard: Linux supports customized keyboards.
-

Advantages and disadvantages of Linux

Advantages of Linux	Disadvantages of Linux
Open Source: Free to use, modify, and distribute.	Learning Curve: Can be difficult for beginners.
Security: Strong security features and less susceptible to malware.	Software Compatibility: Limited availability of proprietary software.
Stability: Highly stable and rarely crashes.	Gaming: Fewer native games compared to Windows.
Performance: Efficient resource usage, runs well on old hardware.	Hardware Compatibility: Some hardware may not be supported.
Customization: Highly customizable to fit various needs.	Technical Support: Less commercial support compared to Windows or macOS.
Community Support: Strong community and extensive online resources.	Professional Software: Limited availability of industry-standard professional software.
Cost: No licensing fees.	Software Installation: Sometimes complex, especially for non-technical users.
Variety: Multiple distributions to choose from.	Driver Availability: Some devices might have limited driver support.
Privacy: Less data collection compared to other OSes.	Gaming Performance: Even with support, performance can be less optimal than on Windows.
Development Environment: Preferred platform for developers due to its support for various programming languages and tools.	Market Share: Lower desktop market share, leading to less mainstream support.

Duties of the System Administrator

1. Installing and Configuring Servers

- Set up new Linux servers, either physical or virtual, ensuring they meet the required specifications and configurations.
- Install necessary software packages and dependencies.

2. Installing and Configuring Application Software

- Deploy and configure application software as required by the organization, including web servers, database servers, and other enterprise applications.
- Regularly apply updates and patches to keep software secure and up to date.

3. Creating and Maintaining User Accounts

- Create, modify, and delete user accounts, setting appropriate permissions and access controls.
- Implement policies for password complexity, expiration, and user roles.

4. Backing Up and Restoring Files

- Develop and implement backup strategies to ensure data integrity and availability.
- Test and perform data restoration to ensure backups are reliable and functional.

5. Monitoring and Tuning Performance

- Resource Management: Adjust system resources such as CPU, memory, and disk I/O to optimize performance.
- Load Balancing: Implement load balancing solutions to distribute workloads evenly across servers.

6. Configuring a Secure System

- Security Policies: Establish and enforce security policies and procedures.

7. Using Tools to Monitor Security

- Deploy intrusion detection systems (IDS) such as Snort or OSSEC to monitor and alert on suspicious activities.
- Set up centralized logging solutions to collect and analyze logs.

Unit 2

Installation of Redhat Linux in virtual machine

1. First, you'll need to download the Red Hat Enterprise Linux ISO image from the Red Hat website. This typically requires an account and subscription.
 2. You can use virtualization software like VirtualBox.
 3. Download and install VirtualBox
 4. Create a New Virtual Machine on VirtualBox.
 5. Assign the amount of RAM. Red Hat recommends at least 2GB (2048 MB), but 4GB (4096 MB) is better for performance.
 6. Create a Virtual Hard Disk.
 7. Choose "Dynamically allocated" to save space on your physical drive and set the size of the virtual hard disk to minimum 20GB.
 8. Configure the Virtual Machine according to your needs.
 9. Now, Install Red Hat Enterprise Linux from the Virtual Machine, configure installation destination and complete the installation.
-

Partitions of Linux OS

1. The partition is the logically separated space on the hard drive created by operating system.
 2. Creating and deleting partitions in Linux is a regular practice because storage devices such as hard drives and USB drives must be structured in some way before they can be used.
 3. In many cases, large storage devices are divided into separate sections called partitions.
 4. The partitioning allows us to divide our hard drive into isolated sections, where each section behaves as its own hard drive.
 5. So, partitioning is particularly helpful if we run multiple operating systems on the drive.
 6. There are many powerful tools available for creating, removing, and otherwise manipulating disk partitions in Linux.
 7. By default, Linux contains the parted, fdisk and cfdisk utilities for disk partitioning.
 8. Differences between parted and the more common fdisk and cfdisk commands include:
 - GPT format: The parted command can create a Globally Unique Identifiers Partition Table (GPT), while fdisk and cfdisk are limited to only DOS partition tables.
 - Larger disks: The DOS partition table can format up to 2TB of disk space, although up to 16TB is possible in some cases. However, a GPT partition table can address up to 8ZiB of space.
 - More partitions: Using primary and extended partitions, DOS partition tables allow us only 16 partitions. With GPT, we get up to 128 partitions by default and can choose to have many more.
 - Reliability: Only one copy of the partition table is stored in a DOS partition. GPT keeps two copies of the partition table (at the beginning and the end of the disk). GPT also uses a CRC checksum to check the partition table integrity, which is not done with DOS partitions.
-

Booting and Shutting down Linux

1. Booting a computer involves loading its operating system.
 2. This starts with the bootstrapping process, where a small piece of code called the bootstrap loader is loaded first, which then loads the operating system.
 3. The bootstrap loader, stored in a fixed location on the hard disk, is small to keep the firmware simple.
 4. In PCs, the BIOS reads the boot sector of the hard disk, containing the bootstrap loader, which then loads the OS.
 5. After Linux is loaded, it initializes hardware and device drivers.
 6. To shut down, all processes are terminated, file systems and swap areas are unmounted, and a message signals that power can be turned off.
 7. Improper shutdowns can result in data loss and an inconsistent, possibly unusable file system.
-

Boot Process in Linux

1. The computer's firmware (BIOS or UEFI) initializes hardware components like the processor, memory, and storage devices.
2. The firmware may also perform a power-on self-test.
3. The BIOS or UEFI selects a boot device based on the system configuration, such as a hard drive, USB drive, or CD drive.
4. The GRUB boot loader is commonly used in modern Linux systems, and its splash screen may appear first when the computer boots.
5. The splash screen may include a menu that allows the user to select the kernel image to use.
6. The Linux kernel decompresses itself, performs hardware checks, and gains access to peripheral hardware.
7. It also initializes and configures the computer's memory and hardware.
8. The kernel loads into memory and initializes key OS services.
9. The boot process is complete when the kernel is initialized and systemd is launched.

Linux File System Structure

- **/:** The root directory is the top-most directory in the Linux file system. All other directories and files are placed under this directory.
 - **/bin:** Contains essential command binaries (executables) needed for the system to boot and run in single-user mode.
 - **/boot:** Holds files needed for the boot process, including the Linux kernel, initial RAM disk image (initrd), and bootloader configuration files (e.g., GRUB).
 - **/dev:** Contains device files, which are special files that represent hardware devices and allow software to interact with the hardware.
 - **/media:** Mount points for removable media such as USB drives, CDs, and DVDs. When you insert a removable media device, it is typically mounted under this directory.
 - **/tmp:** Used to hold temporary files created by system and user processes. Files in this directory are often cleared on reboot.
-

Types of File Systems in linux

- 1. NTFS:** The primary file system for Windows NT and its descendants. Linux provides support for NTFS through the ntfs-3g driver, enabling read and write operations on NTFS-formatted drives.
 - 2. VFAT:** A variant of the FAT file system that supports long filenames. It's commonly used for compatibility with Windows systems and on removable media like USB drives.
 - 3. Ext2 (Second Extended File System):** One of the oldest and most stable file systems in Linux. It lacks journaling, which means it can be faster but less reliable after crashes.
 - 4. XFS:** A high-performance journaling file system created by Silicon Graphics. It's designed for high scalability and efficiency, particularly for large files and large file systems. XFS is known for its speed in handling parallel I/O operations.
 - 5. NFS (Network File System):** Developed by Sun Microsystems, NFS allows a system to share directories and files with others over a network. It enables users to access files as if they were on the local machine.
-

Piping

1. Piping in Linux is a powerful feature that allows the output of one command to be used as the input to another command.
2. This can be used to create complex sequences of operations, allowing for efficient data processing and manipulation directly from the command line.
3. The basic syntax of the pipe command in Linux is as follows:

`command1 | command2`

- `command1`: This is the initial command whose output will serve as input to the next command.
 - `|` (pipe symbol): This symbol is the pipe command itself. It facilitates the transfer of output from `command1` into `command2`.
 - `command2`: This is the subsequent command that receives the output of `command1` as its input.
-

I/O Redirection

1. In Linux, whenever an individual runs a command, it can take input, give output, or do both.
 2. Redirection helps us redirect these input and output functionalities to the files or folders we want, and we can use special commands or characters to do so.
 3. For example, if we run the “date” command, it gives us output on the screen.
 4. However, if we want to save the result in a file instead, we can use output redirection.
 5. This way, we can store the output of the date command in a file and refer to it later.
 6. These redirections can come in handy when we work with multiple and large outputs or inputs since we can use file data directly as input and store results in files.
 7. All this can be done easily on the terminal using some simple commands.
-

RPM

1. RPM, or Red Hat Package Manager, is a command-line tool that manages software packages on Linux systems.
 2. It's the default package manager for Red Hat-based systems, but can also be used on CentOS and Fedora.
 3. RPM packages are standalone binary files that are pre-compiled and built for a specific operating system and hardware architecture.
 4. This makes them advantageous over conventional archive files because they can be installed, updated, or removed independently of each other, and they reduce installation time and CPU usage.
 5. RPM packages, often denoted with the `.rpm` file extension contains all the necessary files, metadata, and scripts required to install and manage software on a Linux system.
-

YUM

1. Yellowdog Updater Modified (YUM) is a free, open-source command-line utility for managing RPM packages on Linux systems.
 2. It was originally developed for Yellow Dog Linux, but is now used by many other Linux distributions, including Red Hat-based systems like CentOS and Fedora.
 3. YUM simplifies software package management by automating tasks like installation, upgrades, and dependency resolution.
 4. While YUM has a command-line interface, other tools can provide graphical user interfaces to its functionality.
 5. Here are some things YUM can do:
 - Install, update, remove, and maintain packages.
 - Automatically handle package dependencies.
 - Download package headers and RPMs from a repository.
 - Access repositories via FTP, HTTP, NFS server, or local filesystem.
 - Configure to use multiple repositories.
-

Unit 3

Bash Shell

1. Bash, short for Bourne-again shell, is a command-line shell and programming language used to control operating systems.
 2. It is the default shell for most Linux distributions and is also used on macOS.
 3. Bash is a powerful tool that can be used to perform a wide variety of tasks, including:
 4. Navigating the file system, Executing commands, Creating and editing files, Automating tasks, and Programming.
 5. To get started with Bash, you can open a terminal window. This is a text-based interface that allows you to interact with your operating system.
 6. Once you have opened a terminal window, you can type Bash commands to perform various tasks.
 7. Bash also allows you to create scripts. Scripts are files that contain a series of Bash commands. When you run a script, the commands in the script are executed one by one.
-

Types of Linux command

Linux have hundreds of commands we can categorized them.

- 1. System Related Commands:** These commands are used to view and manage Linux system-related information,.
 - 2. Hardware Related Commands:** These commands are used to view and manage hardware related aspects of the Linux machine.
 - 3. Statistic Related Commands:** These set of commands are used to view various kinds of stats of the Linux system.
 - 4. User-Related Commands:** These commands are used to manage Linux users.
 - 5. File Related Commands:** These commands are used to handle files and directories.
 - 6. Process Related Commands:** These commands are used to handle Linux processes.
 - 7. File Permission Related Commands:** These commands are used to change permissions of the files.
 - 8. Network Related Commands:** These commands are used to view and edit network configurations related aspects of the system.
-

Basic commands of linux

- ls:** Lists directory contents.
- cd:** Changes the current directory.
- pwd:** Prints the current working directory.
- mkdir:** Creates a new directory.
- rmdir:** Removes an empty directory.
- find:** Searches for files and directories.
- cp:** Copies files or directories.
- mv:** Moves or renames files or directories.
- rm:** Removes files or directories.
- chmod:** Changes file or directory permissions.

Piping

1. Piping in Linux is a powerful feature that allows the output of one command to be used as the input to another command.
2. This can be used to create complex sequences of operations, allowing for efficient data processing and manipulation directly from the command line.
3. The basic syntax of the pipe command in Linux is as follows:

`command1 | command2`

- `command1`: This is the initial command whose output will serve as input to the next command.
 - `|` (pipe symbol): This symbol is the pipe command itself. It facilitates the transfer of output from `command1` into `command2`.
 - `command2`: This is the subsequent command that receives the output of `command1` as its input.
-

I/O Redirection

1. In Linux, whenever an individual runs a command, it can take input, give output, or do both.
 2. Redirection helps us redirect these input and output functionalities to the files or folders we want, and we can use special commands or characters to do so.
 3. For example, if we run the “date” command, it gives us output on the screen.
 4. However, if we want to save the result in a file instead, we can use output redirection.
 5. This way, we can store the output of the date command in a file and refer to it later.
 6. These redirections can come in handy when we work with multiple and large outputs or inputs since we can use file data directly as input and store results in files.
 7. All this can be done easily on the terminal using some simple commands.
-

RPM

1. RPM, or Red Hat Package Manager, is a command-line tool that manages software packages on Linux systems.
 2. It's the default package manager for Red Hat-based systems, but can also be used on CentOS and Fedora.
 3. RPM packages are standalone binary files that are pre-compiled and built for a specific operating system and hardware architecture.
 4. This makes them advantageous over conventional archive files because they can be installed, updated, or removed independently of each other, and they reduce installation time and CPU usage.
 5. RPM packages, often denoted with the `.rpm` file extension contains all the necessary files, metadata, and scripts required to install and manage software on a Linux system.
-

YUM

1. Yellowdog Updater Modified (YUM) is a free, open-source command-line utility for managing RPM packages on Linux systems.
 2. It was originally developed for Yellow Dog Linux, but is now used by many other Linux distributions, including Red Hat-based systems like CentOS and Fedora.
 3. YUM simplifies software package management by automating tasks like installation, upgrades, and dependency resolution.
 4. While YUM has a command-line interface, other tools can provide graphical user interfaces to its functionality.
 5. Here are some things YUM can do:
 - Install, update, remove, and maintain packages.
 - Automatically handle package dependencies.
 - Download package headers and RPMs from a repository.
 - Access repositories via FTP, HTTP, NFS server, or local filesystem.
 - Configure to use multiple repositories.
-

Unit 4

Managing Users in Linux

1. A user is an entity, in a Linux operating system, that can manipulate files and perform several other operations.
 2. Each user is assigned an ID that is unique for each user in the operating system.
 3. After installation of the operating system, the ID 0 is assigned to the root user and the IDs 1 to 999 (both inclusive) are assigned to the system users and hence the ids for local user begins from 1000 onwards.
 4. In a single directory, we can create 60,000 users.
-

Commands for User Management

- `useradd username`: Add a new user.
 - `passwd username`: Set or change the password for a user.
 - `userdel username`: Delete a user.
 - `groupadd groupname`: Add a new group.
 - `usermod -aG groupname username`: Add a user to a group.
 - `usermod -g groupname username`: Change a user's primary group.
 - `groupdel groupname`: Delete a group.
 - `getent group groupname`: View detailed group information.
-

Managing Passwords in Linux

1. To access the system, a user needs a password.
 2. To enable these users, assign passwords using the passwd command.
 3. A user can use passwd command to change his password.
 4. The passwd command will first prompt for the old password and then for the new one.
 5. The root user can change passwords as well. For example, "passwd parag" would change the password for user parag.
 6. The user root can use the passwd command in three generic ways.
 7. First, you can use it to change a password.
 8. Second, it can also be used to set password expiry information, which dictates that a password will expire at a particular date.
 9. Lastly, the passwd command can be used where an administrator can use passwd to lock an account so that login is disabled temporarily.
-

External authentication

1. External authentication sources allow Linux systems to authenticate users against external directories like LDAP or Active Directory.
2. This centralizes user management, simplifies administration, and enhances security by enabling consistent password policies and account management across multiple systems.
3. Here are some external authentication sources that can be used on Linux:
 - **LDAP (Lightweight Directory Access Protocol):** A common authentication mechanism that allows authentication against a directory of usernames and passwords maintained by an LDAP server.
 - **Microsoft Active Directory:** Used to authenticate devices such as Windows or Linux computers on a network.
 - **OpenLDAP:** A popular open source LDAP server that provides core directory services to resources that leverage the LDAP protocol.

Pluggable Authentication Modules

1. Programs that grant users access to a system use authentication to verify each other's identity.
 2. In Red Hat Enterprise Linux, many programs are configured to use a centralized authentication mechanism called Pluggable Authentication Modules (PAM).
 3. PAM uses a pluggable, modular architecture, which affords the system administrator a great deal of flexibility in setting authentication policies for the system.
 4. PAM is a useful system for developers and administrators for several reasons:
 5. PAM provides a common authentication scheme that can be used with a wide variety of applications.
 6. PAM provides significant flexibility and control over authentication for both system administrators and application developers.
 7. PAM provides a single, fully-documented library which allows developers to write programs without having to create their own authentication schemes.
-

Basic Permissions: Read, Write, and Execute

1. In Linux, file permissions are represented using three types of permissions: read (r), write (w), and execute (x).
2. These permissions can be set for three different categories of users: the owner of the file, the group to which the file belongs, and others.
3. When Linux file permissions are represented by numbers, it's called numeric mode.
4. In numeric mode, a three-digit value represents specific file permissions.
5. These are called octal values.
6. The first digit is for owner permissions, the second digit is for group permissions, and the third is for other users.
7. Each permission has a numeric value assigned to it:

r (read): 4

w (write): 2

x (execute): 1

8. For example, a file might have read, write, and execute permissions for its owner, and only read permission for all other users.

Owner: $rw x = 4 + 2 + 1 = 7$

Group: $r - - = 4 + 0 + 0 = 4$

Others: $r - - = 4 + 0 + 0 = 4$

9. The results produce the three-digit value 744.
-

Setting Default Permissions with umask

1. In Linux, umask (user file-creation mode mask) determines the default permissions for newly created files and directories.
 2. The umask value is subtracted from the system's default permissions to set the actual default permissions.
 3. The default permissions for a new file are typically 666 (read and write for everyone).
 4. The default permissions for a new directory are typically 777 (read, write, and execute for everyone).
 5. The umask value is subtracted from the default permissions to determine the final permissions for new files and directories.
 6. For example, if the umask is set to 022:
 7. New files will have permissions $666 - 022 = 644$ (rw-r--r--).
 8. New directories will have permissions $777 - 022 = 755$ (rwxr-xr-x).
-

File Attributes

Attributes define properties of files. The file can have basic attribute such as File Index, Owner, File Size, File Time Stamps etc.

The files and directories can have following attributes:

- **a - append only:** this attribute allows a file to be added to, but not to be removed. It prevents accidental or malicious changes to files that record data, such as log files.
 - **c - Compressed:** it causes the kernel to compress data written to the file automatically and uncompress it when it's read back.
 - **i - Immutable:** it makes a file immutable.
 - **j - Data journaling:** it ensures that on an Ext3 file system the file is first written to the journal and only after that to the data blocks on the hard disk.
 - **s - Secure deletion:** it makes sure that recovery of a file is not possible after it has been deleted.
-

Unit 5

IP networking

1. IP networking, or Internet Protocol networking, is the use of IP addresses to connect devices on a network.
 2. IP addresses are unique to each device and are a set of rules that structure and format data sent over the internet.
 3. IP networking is essential for digital communication, allowing information to flow between devices and supporting activities like browsing the web, streaming videos, and sending emails.
 4. IP networks can be used in both homes and businesses.
 5. In a business setting, an IP network can help employees collaborate and communicate efficiently, while keeping viruses and hackers out with a private network.
 6. In a home setting, an IP network can enable residential Wi-Fi.
-

TCP Networking

1. Transmission Control Protocol (TCP) is a communications standard that allows devices and applications to exchange messages over a network.
 2. It's a transport protocol that's built on top of IP to ensure reliable packet transmission.
 3. TCP includes mechanisms to solve problems that can arise from packet-based messaging, such as lost, out of order, duplicate, or corrupted packets.
 4. It uses sequence numbers to synchronize itself with the remote host.
 5. TCP also wraps each data packet with a header that contains 10 mandatory fields, totaling 20 bytes, or octets.
 6. Each header holds information about the connection and the current data being sent, such as the source port and the destination port.
-

Network classes

Network classes refer to the categorization of IP address ranges used in network design and management. The primary classes are A, B, C, D, and E, each serving different purposes and having distinct address ranges. Here's a breakdown of each class:

Class A

- Use Case: Designed for very large networks with a vast number of hosts. Each Class A network can support approximately 16 million hosts.
- Network Portion: The first octet (8 bits) is the network portion.
- Host Portion: The remaining three octets (24 bits) are the host portion.

Class B

- Use Case: Suitable for medium-sized networks. Each Class B network can support around 65,000 hosts.
- Network Portion: The first two octets (16 bits) are the network portion.
- Host Portion: The remaining two octets (16 bits) are the host portion.

Class C

- Use Case: Ideal for smaller networks. Each Class C network can support 254 hosts.
- Network Portion: The first three octets (24 bits) are the network portion.
- Host Portion: The last octet (8 bits) is the host portion.

Class D

- Use Case: Reserved for multicast groups. Not used for typical network host addressing.

Class E

- Use Case: Reserved for experimental purposes. Not used for typical network host addressing.
-

Network interface card (NIC)

1. A network interface card (NIC) is a hardware component without which a computer cannot be connected over a network.
 2. It is a circuit board installed in a computer that provides a dedicated network connection to the computer.
 3. It is also called network interface controller, network adapter, or LAN adapter.
 4. NIC allows both wired and wireless communications.
 5. NIC allows communications between computers connected via local area network (LAN) as well as communications over large-scale network through Internet Protocol (IP).
 6. NIC is both a physical layer and a data link layer device, i.e. it provides the necessary hardware circuitry so that the physical layer processes and some data link layer processes can run on it.
-

Subnetting

1. Subnetting is a process used in IP networking to divide a larger network into smaller, more manageable sub-networks, or subnets.
 2. Subnetting reduces the size of broadcast domains, minimizing the broadcast traffic in each subnet and improving overall network performance.
 3. Subnetting allows network administrators to segment a network into smaller, isolated subnets. This helps in containing network breaches within a single subnet, preventing the spread of malicious activity.
 4. Subnetting allows for more efficient allocation of IP addresses by matching subnet sizes to the number of required hosts, reducing wastage of IP addresses.
 5. Subnetting can help in distributing network traffic across multiple subnets, balancing the load and preventing network congestion.
 6. Subnetting supports scalable network design, allowing organizations to grow their network in a structured manner. New subnets can be created as needed without disrupting the existing network.
-

Subnet example

You have a network with the IP address 192.168.1.0/24 and you need to create 4 subnets.

Step 1: Determine the Subnet Mask

- **Original Network:** 192.168.1.0/24
- **Subnet Mask:** 255.255.255.0 (or /24 in CIDR notation)
- **Number of Subnets Required:** 4

To find the new subnet mask, calculate how many bits you need to borrow from the host portion of the address to create the required number of subnets.

$2^n \geq \text{Number of Subnets}$

$2^2 = 4$, so you need to borrow 2 bits.

Step 2: Calculate the New Subnet Mask

- **Original Subnet Mask in Binary:** 11111111.11111111.11111111.00000000 (/24)
- **New Subnet Mask:** Borrow 2 bits
- **New Subnet Mask in Binary:** 11111111.11111111.11111111.11000000 (/26)
- **New Subnet Mask in Decimal:** 255.255.255.192

Step 3: Determine the Subnet Addresses

With a /26 subnet mask, each subnet will have 64 addresses ($2^6 = 64$).

- **First Subnet:** 192.168.1.0/26
 - **Range:** 192.168.1.0 - 192.168.1.63
 - **Network Address:** 192.168.1.0
 - **Broadcast Address:** 192.168.1.63
- **Second Subnet:** 192.168.1.64/26
 - **Range:** 192.168.1.64 - 192.168.1.127
 - **Network Address:** 192.168.1.64
 - **Broadcast Address:** 192.168.1.127
- **Third Subnet:** 192.168.1.128/26

- **Range:** 192.168.1.128 - 192.168.1.191
 - **Network Address:** 192.168.1.128
 - **Broadcast Address:** 192.168.1.191
- **Fourth Subnet:** 192.168.1.192/26
 - **Range:** 192.168.1.192 - 192.168.1.255
 - **Network Address:** 192.168.1.192
 - **Broadcast Address:** 192.168.1.255

Step 4: Assign IP Addresses

Each subnet can now be used to assign IP addresses to devices. For example, in the first subnet (192.168.1.0/26):

- **Usable IP Range:** 192.168.1.1 - 192.168.1.62 (excluding the network address and broadcast address)

Summary:

By subnetting the 192.168.1.0/24 network into 4 subnets, you have created:

- 192.168.1.0/26
- 192.168.1.64/26
- 192.168.1.128/26
- 192.168.1.192/26

Each subnet has 64 addresses, with 62 usable for hosts.

This example illustrates how subnetting divides a larger network into smaller subnets, optimizing IP address usage and improving network management.

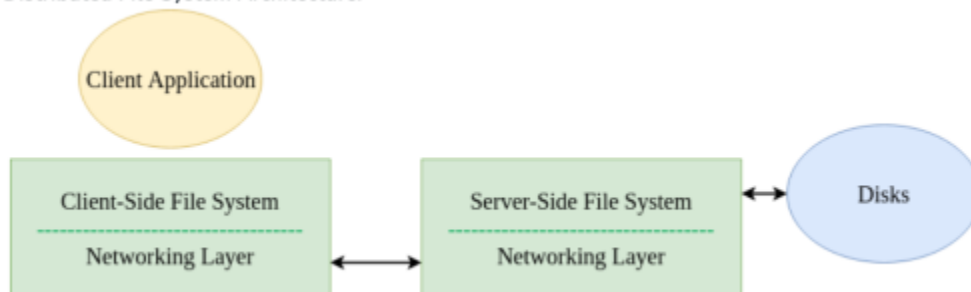
DHCP

1. Dynamic Host Configuration Protocol, is a network protocol used to automate the process of assigning IP addresses and other network configuration parameters to devices (such as computers, smartphones, and printers) on a network.
 2. DHCP helps in managing the entire process automatically and centrally.
 3. DHCP helps in maintaining a unique IP Address for a host using the server.
 4. DHCP servers maintain information on TCP/IP configuration and provide configuration of address to DHCP-enabled clients in the form of a lease offer.
 5. IP Address Pool is the pool or container of IP Addresses possessed by the DHCP Server.
 6. It has a range of addresses that can be allocated to devices.
 7. DHCP servers can also provide information about the default gateway, which is the device that packets are sent to when the destination is outside the local network.
-

Network File System architecture

1. The architecture consists of a client-side file system and a server-side file system.
2. A client application issues a system call (e.g. read(), write(), open(), close() etc.) to access files on the client-side file system, which in turn retrieves files from the server.
3. For instance, assume that a client application issues the read() system call.
4. The client-side file system then messages the server-side file system to read a block from the server's disk and return the data back to the client.
5. Finally, it buffers this data into the read() buffer and completes the system call.
6. The server-side file system is also simply called the file server.
7. NFS Allows easy sharing of data among clients.
8. NFS Provides centralized administration.
9. An NFS server may have multiple file systems or partitions.
10. The volume identifier tells the server which file system is being referred to.

Distributed File System Architecture:



Security measures in NFS

1. NFSv4 and Kerberos: NFSv4 introduces built-in support for Kerberos authentication, which provides strong authentication mechanisms between NFS clients and servers. Kerberos uses tickets to authenticate users, ensuring that only authorized users can access NFS resources.

2. Port Restrictions: NFS uses several ports for communication between clients and servers. Configuring firewalls to allow only necessary ports from trusted networks or using VPNs/SSH tunnels ensures that NFS traffic remains secure and protected from unauthorized access.

3. Log Monitoring: Regularly monitoring NFS logs helps in detecting and responding to suspicious activities or potential security incidents.

4. File Permissions: Properly setting file system permissions ensures that only authorized users or groups have access to NFS-shared directories and files.

5. System Updates: Keeping NFS server and client systems updated with the latest security patches and updates helps mitigate known vulnerabilities and ensures a secure environment.

Unit 6

DNS

1. DNS stands for Domain Name System, and it's an internet service that translates domain names into IP addresses.
 2. Domain names are human-readable, like "www.amazon.com", while IP addresses are machine-readable, like "192.0.2.44".
 3. DNS acts like a phonebook for the internet, allowing users to access information online through domain names like "nytimes.com" or "espn.com".
 4. When a user types a domain name into their browser, DNS finds the corresponding IP address so the browser can load the internet page.
 5. Caching makes DNS more efficient, allowing servers to respond quickly when the same IP address is requested again.
 6. For example, if everyone in an office needs to access the same training video on a particular website on the same day, the local DNS server only needs to resolve the name once, and then it can serve all the other requests from its cache.
-

DNS hierarchy

DNS (Domain Name System) hierarchy is a structured model that organizes the domain name space of the internet in a hierarchical manner. It ensures that domain names are mapped to their corresponding IP addresses in an organized and scalable way. The DNS hierarchy is divided into several levels:

1. Root Level:

- At the top of the DNS hierarchy is the root level, represented by a single dot (".").
- The root level is managed by the Internet Assigned Numbers Authority (IANA).
- It consists of root servers that contain information about the top-level domain (TLD) servers.

2. Top-Level Domains (TLDs):

- Below the root level are the top-level domains, such as .com, .org, .net, .edu, .gov, and country-code TLDs like .uk, .de, .jp.
- Each TLD has its own set of authoritative name servers that store information about second-level domains within that TLD.

3. Second-Level Domains:

- Second-level domains are directly below the TLDs. For example, in "example.com," "example" is the second-level domain.
- These domains are typically managed by individuals, organizations, or businesses.
- The authoritative name servers for these domains contain information about subdomains and hostnames under the second-level domain.

4. Subdomains:

- Subdomains are created to organize different sections or services of a website. For example, "blog.example.com" is a subdomain of "example.com."
 - Each subdomain can have its own DNS records and be managed independently.
-

Types of DNS Servers

DNS (Domain Name System) servers are categorized based on their roles and functions within the DNS hierarchy. Here are the main types of DNS servers:

1. Root Name Servers:

- These servers are at the top of the DNS hierarchy and contain the information necessary to locate the top-level domain (TLD) name servers.

2. Top-Level Domain (TLD) Name Servers:

- These servers manage the information for each top-level domain (e.g., .com, .org, .net, country-code TLDs like .uk and .jp).

3. Authoritative Name Servers:

These servers contain the actual DNS records for a domain.

There are two types of authoritative name servers:

- Primary (Master) DNS Server: The main server where the DNS records are stored and maintained.
- Secondary (Slave) DNS Server: A backup server that obtains a copy of the DNS records from the primary server for redundancy and load balancing.

4. Recursive Resolver:

- Also known as a DNS resolver, this server is responsible for receiving DNS queries from client devices (e.g., computers, smartphones) and resolving them by making a series of requests to other DNS servers.

5. Forwarding DNS Server:

- This server acts as an intermediary, forwarding DNS queries from client devices to an external recursive resolver.

6. Caching DNS Server:

- This server temporarily stores the results of DNS queries it has resolved, reducing the need to repeatedly query upstream DNS servers for the same information.

How They Work Together

1. **Client Device:** When a client device needs to resolve a domain name, it sends a DNS query to a recursive resolver.
 2. **Recursive Resolver:** The recursive resolver checks its cache to see if it already has the answer. If not, it starts the query process.
 3. **Root Name Server:** The recursive resolver queries a root name server to find out which TLD name server is responsible for the requested domain.
 4. **TLD Name Server:** The recursive resolver then queries the TLD name server to find out which authoritative name server holds the DNS records for the domain.
 5. **Authoritative Name Server:** The recursive resolver finally queries the authoritative name server to get the actual DNS records for the domain.
 6. **Client Device:** The recursive resolver returns the resolved IP address or other DNS records to the client device, which can then access the requested resource.
-

Types of DNS Zones

DNS zones are distinct portions of the domain namespace that are managed separately. They contain DNS records for the domains and subdomains within them. Here are the main types of DNS zones:

1. Primary (Master) Zone:

- A primary zone is where the original source of the DNS data is stored.
- It is managed by the primary (master) DNS server.
- The zone file in a primary zone is editable, and any changes are made directly on the primary server.
- Example: The DNS zone file for facebook.com would contain all the DNS records for the domain facebook.com.

2. Secondary (Slave) Zone:

- A secondary zone is a read-only copy of the primary zone.
- Secondary servers obtain zone data from the primary server through a process called zone transfer.
- These zones provide redundancy and load balancing by distributing DNS queries across multiple servers.
- Example: If facebook.com has a secondary zone on another server, this server would have an identical copy of the DNS records from the primary zone but would not be able to modify them.

3. Stub Zone:

- A stub zone contains only the essential records needed to identify the authoritative name servers for a specific zone.
- Stub zones are used to maintain information about delegated zones without holding a complete copy of the zone data.
- Example: If facebook.com delegates sub.facebook.com to another set of name servers, a stub zone can be created to keep track of the authoritative servers for sub.facebook.com.

4. Forward Zone:

- A forward zone is used to forward DNS queries for specific domains to another DNS server instead of resolving them locally.

5. Reverse Zone:

- A reverse zone is used to map IP addresses back to domain names, the opposite of the usual forward DNS resolution.
 - It contains PTR (pointer) records that provide the domain name associated with an IP address.
 - Example: An IP address like 192.0.2.1 might map back to a domain name like host1.facebook.com in a reverse zone.
-

Unit 7

Samba Server

1. Samba is an open-source software suite that allows Linux, Windows, and Unix systems to share files, printers, and other resources over a network.
 2. It uses the SMB/CIFS protocol and various authentication methods to bridge the gap between different operating systems.
 3. Samba can be used in mixed-OS environments, such as when you want to access your desktop files from a laptop or share files with Windows and macOS users.
 4. Samba can also integrate Linux/Unix servers and desktops into Active Directory environments, and can function as a domain controller or a regular domain member.
 5. It gives network administrators flexibility in setup, configuration, and choice of systems and equipment.
 6. Samba supports various security mechanisms, including user-level security, share-level security, and integration with Kerberos for secure authentication.
-

Setting up a Samba file share service

1. Install the Samba packages
 2. Create a directory and provide the required permissions
 3. Configure the shared directory
 4. Create a Samba user account
 5. Configure SELinux on the /share directory
 6. Enable and start the smb and nmb services
 7. Configure firewall to allow Samba traffic
-

Samba Daemons

1. smbd (Server Message Block Daemon)

- **Role:** The smbd daemon provides file and print sharing services to SMB/CIFS clients. It handles all the core functionalities of Samba, including authentication, authorization, file sharing, and print services.
- **Functions:**
 - **File Sharing:** Manages access to shared files and directories, including handling file reads, writes, and modifications.
 - **Print Services:** Manages access to shared printers.
 - **Authentication:** Authenticates users attempting to access the shared resources using various backends (e.g., local passwords, LDAP).
 - **Authorization:** Enforces access control rules and permissions for users and groups.

2. nmbd (NetBIOS Name Server Daemon)

- **Role:** The nmbd daemon handles NetBIOS name resolution and browsing. It enables Samba to interact with the Windows network environment by providing name resolution and browsing services.
- **Functions:**
 - **NetBIOS Name Resolution:** Converts NetBIOS names (used by Windows) to IP addresses. This is crucial for locating services and machines within a Windows network.
 - **Browsing:** Maintains the list of available SMB/CIFS services and servers in the network, allowing users to browse and discover network resources.

3. winbindd (Windows Name Service Switch Daemon)

- **Role:** The winbindd daemon provides unified login and directory services by integrating Unix/Linux systems with Windows Active Directory (AD) environments. It allows Unix/Linux systems to use Windows accounts and groups for authentication and authorization.
 - **Functions:**
 - **User and Group Mapping:** Maps Windows users and groups to Unix/Linux counterparts, enabling consistent identity management across platforms.
 - **Authentication:** Allows Unix/Linux systems to authenticate users against a Windows domain or Active Directory.
 - **Name Service Switch (NSS) Integration:** Integrates with the Unix/Linux NSS to provide user and group information from a Windows domain, making Windows accounts available to Unix/Linux applications and services.
 - **PAM Integration:** Integrates with the Pluggable Authentication Modules (PAM) system to provide authentication services for Unix/Linux applications.
-

Postfix

1. Postfix is composed of several independent modules, each responsible for a specific task such as receiving mail, routing it, and delivering it.
2. The master daemon controls all the processes and is responsible for starting and stopping Postfix services.
3. Postfix receives incoming mail through the SMTP protocol. The smtpd daemon listens on port 25 for incoming SMTP connections.
4. Once received, the mail is stored in the mail queue until it can be delivered. The queue manager oversees this process.
5. Postfix can deliver mail to local mailboxes, forward it to other mail servers, or send it to remote recipients using SMTP.
6. It uses multiple layers of security checks, including access control, content filtering, and spam protection.
7. Postfix is also optimized for performance, capable of handling high volumes of mail traffic efficiently.

Unit 8

firewall

1. A firewall is a network security device that monitors incoming and outgoing network traffic and decides whether to allow or block specific traffic based on a defined set of security rules.
 2. Firewalls are the first line of defense in network security.
 3. They establish a barrier between secured and controlled internal networks that can be trusted and untrusted outside networks, such as the Internet.
 4. A firewall can be hardware, software, software-as-a service (SaaS), public cloud, or private cloud (virtual).
 5. Firewalls can be set up to block traffic linked to known malware or other security concerns, assisting in the defense against these kinds of attacks.
 6. Many industries are bound by rules that demand the usage of firewalls or other security measures.
-

Working of Firewall

1. Incoming Packet: A data packet arrives at the firewall from an external source (e.g., the Internet) or an internal source (e.g., a computer within the network).

2. Header Inspection: The firewall inspects the packet header to determine its source and destination IP addresses, port numbers, and protocol (e.g., TCP, UDP).

3. Rule Checking: The firewall compares the packet information against its predefined security rules. These rules are defined by the network administrator and determine whether the packet should be allowed or blocked.

4. State Table Check: For stateful firewalls, the firewall checks its state table to determine if the packet is part of an established connection or a legitimate connection request.

5. Allow or Block: Based on the rules and inspection results, the firewall decides whether to allow or block the packet.

- **Allowed Packet:** If the packet matches an "allow" rule and passes all inspections, it is permitted to proceed to its destination.
- **Blocked Packet:** If the packet matches a "block" rule or fails any inspection, it is discarded, and optionally, a log entry is created.

6. Address Translation: If Network Address Translation (NAT) is enabled, the firewall translates private internal IP addresses to public IP addresses (and vice versa) before forwarding the packet. This provides an additional layer of security by hiding internal network addresses.

7. Forwarding: The firewall forwards the allowed packet to its intended destination, whether it's an internal device or an external server.

Types of Firewalls

1. Packet-Filtering Firewalls:

- **Function:** Inspect packets (small chunks of data) that are sent over the network.
- **Criteria:** Based on predefined rules, it allows or blocks packets based on their source and destination IP addresses, port numbers, and protocols.
- **Strengths:** Simple and fast.
- **Weaknesses:** Limited in capability as it does not inspect the packet contents.

2. Stateful Inspection Firewalls:

- **Function:** Monitors the state of active connections and makes decisions based on the context of the traffic.
- **Criteria:** Keeps track of the state of connections (e.g., TCP connections) and allows only packets that are part of an established connection or are legitimate connection requests.
- **Strengths:** More secure as it understands the state of connections.
- **Weaknesses:** More resource-intensive than packet-filtering firewalls.

3. Proxy Firewalls (Application-Level Gateways):

- **Function:** Intercepts all messages entering and leaving the network and effectively hides the true network addresses.
- **Criteria:** Acts as an intermediary for requests from clients seeking resources from other servers, and inspects the data at the application layer.
- **Strengths:** Provides extensive security as it can inspect the payload and content of the traffic.
- **Weaknesses:** Can slow down network performance due to the deep inspection of packets.

4. Next-Generation Firewalls (NGFWs):

- **Function:** Incorporates traditional firewall capabilities with additional features like deep packet inspection, intrusion prevention systems (IPS), and the ability to inspect encrypted traffic.
- **Criteria:** Uses advanced techniques to detect and prevent sophisticated attacks.
- **Strengths:** Highly effective in dealing with modern threats.
- **Weaknesses:** Expensive and complex to manage.

5. Unified Threat Management (UTM) Firewalls:

- **Function:** Combines multiple security features such as firewall, antivirus, intrusion detection/prevention, and content filtering into a single device.
 - **Criteria:** Provides comprehensive protection by integrating multiple security functions.
 - **Strengths:** Simplifies security management.
 - **Weaknesses:** Can become a single point of failure if not properly managed.
-

iptables in Linux

iptables is a powerful and flexible tool for managing network packet filtering, NAT (Network Address Translation), and other packet processing tasks in Linux. It is part of the Netfilter framework in the Linux kernel.

Overview of Linux iptables

1. Tables: iptables operates on different tables, each designed for specific types of packet processing tasks. The main tables are:

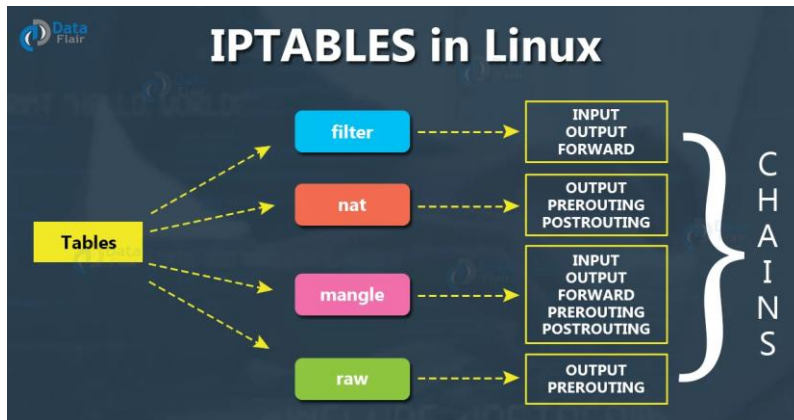
- **filter:** The default table for filtering packets (i.e., deciding whether to allow or block packets).
- **nat:** Used for Network Address Translation (NAT), which modifies packet source/destination addresses.
- **mangle:** Used for specialized packet alteration.
- **raw:** Used for configuring exemptions from connection tracking.
- **security:** Used for Mandatory Access Control (MAC) rules.

2. Chains: Each table contains a set of predefined chains, which are lists of rules that packets traverse. The main predefined chains are:

- **PREROUTING:** Packets are processed before routing decisions are made.
- **INPUT:** Packets destined for the local system.
- **FORWARD:** Packets routed through the system (not destined for the local system).
- **OUTPUT:** Packets generated by the local system.
- **POSTROUTING:** Packets processed after routing decisions have been made.

3. Rules:

- Rules are the basic building blocks of iptables. Each rule specifies conditions that packets must meet and the corresponding action to take if the conditions are met.
- Actions (Targets): Common actions include ACCEPT (allow the packet), DROP (discard the packet), and REJECT (discard the packet and send an error response).



Masquerading

1. Masquerading, often referred to as Network Address Translation (NAT), is a technique used in networking to modify network address information in IP packet headers.
 2. This is typically used to hide an internal network's IP addresses by replacing them with the IP address of the NAT router.
 3. NAT can be used in conjunction with firewall rules to enhance security, only allowing specific traffic through.
 4. Masquerading allows multiple devices on a local network to share a single public IP address, which is crucial in environments where public IP addresses are limited.
 5. Simplifies network management by providing a single point of access to the internet, which can be easier to manage and monitor.
 6. Can handle a large number of simultaneous connections, making it suitable for small to medium-sized networks.
-

Different types of NAT

1. Static NAT (SNAT)

- **Description:** Maps a single private IP address to a single public IP address. This is a one-to-one mapping.
- **Use Case:** Useful when an internal server needs to be accessible from the internet using a specific public IP address.
- **Example:** A web server with a private IP address 192.168.1.10 is mapped to a public IP address 203.0.113.10.

2. Dynamic NAT (DNAT)

- **Description:** Maps a private IP address to a public IP address from a pool of public addresses. This is also a one-to-one mapping, but the public IP is dynamically assigned from a pool.
- **Use Case:** Useful when the number of private IP addresses is less than or equal to the number of available public IP addresses.
- **Example:** Internal devices are assigned public IPs from a pool (e.g., 203.0.113.1 to 203.0.113.10) as they connect to the internet.

3. Port Address Translation (PAT) / Overloading / Masquerading

- **Description:** Maps multiple private IP addresses to a single public IP address but differentiates each connection using a unique port number. This is a many-to-one mapping.
- **Use Case:** Commonly used in home networks and small offices where multiple devices need to share a single public IP address.
- **Example:** Multiple devices (e.g., 192.168.1.2, 192.168.1.3) connect to the internet using the same public IP address (e.g., 203.0.113.1) but with different port numbers.

4. Destination NAT (DNAT)

- **Description:** Changes the destination IP address of incoming packets. Often used to forward traffic to an internal server.
- **Use Case:** Useful for services like web servers or email servers that need to be accessible from the outside world.
- **Example:** Incoming traffic to 203.0.113.20 on port 80 is forwarded to an internal server at 192.168.1.20.

5. Source NAT (SNAT)

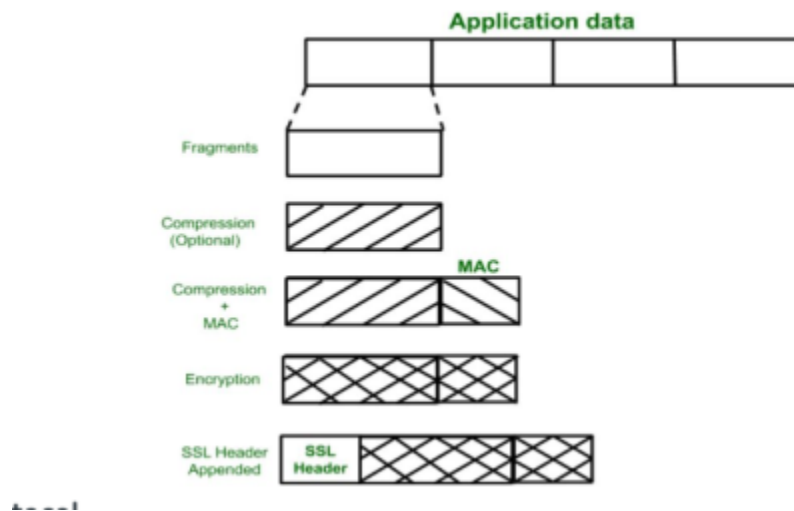
- **Description:** Changes the source IP address of outgoing packets. Often used for outgoing traffic.
 - **Use Case:** Useful for ensuring that outgoing traffic appears to come from a specific public IP address.
 - **Example:** Outgoing traffic from an internal server (192.168.1.30) is modified to appear as if it is coming from 203.0.113.30.
-

Apache

1. Apache HTTP Server is a free and open-source web server that delivers web content through the internet.
 2. It is commonly referred to as Apache and after development, it quickly became the most popular HTTP client on the web.
 3. Apache is just one component that is needed in a web application stack to deliver web content.
 4. One of the most common web application stacks involves LAMP, or Linux, Apache, MySQL, and PHP.
 5. Linux is the operating system that handles the operations of the application.
 6. Apache is the web server that processes requests and serves web assets and content via HTTP.
 7. MySQL is the database that stores all your information in an easily queried format.
 8. PHP is the programming language that works with apache to help create dynamic web content.
-

Secure Sockets Layer

1. SSL, or Secure Sockets Layer, is an Internet security protocol that encrypts data to keep it safe.
2. SSL encrypts data transmitted over the web, ensuring privacy.
3. If someone intercepts the data, they will see only a jumble of characters that is nearly impossible to decode.
4. SSL starts an authentication process called a handshake between two devices to confirm their identities, making sure both parties are who they claim to be.
5. SSL digitally signs data to ensure it hasn't been tampered with, verifying that the data received is exactly what was sent by the sender.
6. In the SSL Record Protocol application data is divided into fragments.
7. The fragment is compressed and then encrypted MAC (Message Authentication Code) generated by algorithms like SHA (Secure Hash Protocol) and MD5 (Message Digest).
8. After that encryption of the data is done and in last SSL header is appended to the data.



---END---

